

```

import os

import logging

import random

import requests

import json

from datetime import datetime

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, InputMediaPhoto

from telegram.ext import Application, CommandHandler, CallbackQueryHandler, ContextTypes

from apscheduler.schedulers.asyncio import AsyncIOScheduler


logging.basicConfig(level=logging.INFO)

logger = logging.getLogger(__name__)


# — Конфиг из переменных окружения —————
TELEGRAM_TOKEN = os.environ["TELEGRAM_TOKEN"]

ADMIN_CHAT_ID = int(os.environ["ADMIN_CHAT_ID"])

CHANNEL_ID = os.environ["CHANNEL_ID"] # например @mychannel или -100123456

PIXABAY_API_KEY = os.environ["PIXABAY_API_KEY"]

ANTHROPIC_API_KEY = os.environ["ANTHROPIC_API_KEY"]


# — Темы для поиска контента —————
TOPICS = [
    {"ru": "производство бумаги", "en": "paper production factory"},
    {"ru": "производство картона", "en": "cardboard manufacturing"},
    {"ru": "макулатура переработка", "en": "paper recycling waste"},
    {"ru": "производство из макулатуры", "en": "recycled paper production"},
    {"ru": "гофрокартон", "en": "corrugated cardboard"},
    {"ru": "гофроящики упаковка", "en": "corrugated boxes packaging"},
    {"ru": "гофропроизводство завод", "en": "corrugated box factory"},
]

```

```

HOLIDAY_TOPICS = [
    {"ru": "День работника леса", "en": "forestry worker holiday"},
    {"ru": "День эколога", "en": "ecology environment day"},
    {"ru": "День переработки", "en": "recycling day environment"},
    {"ru": "День бухгалтера России", "en": "Russian Accountant's Day"},
    {"ru": "День энергетика в России", "en": "Russian Power Engineer's Day"},
    {"ru": "День работников отдела кадров в России", "en": "Russia HR Professionals' Day"},
    {"ru": "Рабочий юмор", "en": "Humor at work "},
]

```

— Pixabay: поиск изображения

```

def search_image(query_en: str) -> dict | None:
    url = "https://pixabay.com/api/"
    params = {
        "key": PIXABAY_API_KEY,
        "q": query_en,
        "image_type": "photo",
        "orientation": "horizontal",
        "min_width": 800,
        "safesearch": "true",
        "per_page": 20,
        "lang": "en",
    }
    try:
        r = requests.get(url, params=params, timeout=10)
        data = r.json()
        hits = data.get("hits", [])
        if not hits:

```

```

        return None

    # Берём случайное из топ-10
    hit = random.choice(hits[:10])

    return {
        "url": hit["largeImageURL"],
        "tags": hit.get("tags", ""),
        "user": hit.get("user", ""),
        "page": hit.get("pageURL", ""),
    }

except Exception as e:
    logger.error(f"Pixabay error: {e}")

    return None

```

— Claude: генерация подписи

```

def generate_caption(topic_ru: str, image_tags: str) -> str:

    headers = {
        "x-api-key": ANTHROPIC_API_KEY,
        "anthropic-version": "2023-06-01",
        "content-type": "application/json",
    }

    prompt = (
        f"Напиши короткую подпись (2-4 предложения) для поста в Telegram-канале о бумажной и "
        f"картонной промышленности. "
        f"Тема поста: «{topic_ru}». "
        f"На фото: {image_tags}. "
        f"Стиль: информативно, профессионально, без лишнего пафоса. "
        f"Добавь 3-5 тематических хэштегов в конце. "
        f"Отвечай только текстом подписи, без вступлений. "
    )

    body = {
        "model": "claude-sonnet-4-6",
        "max_tokens": 300,
        "messages": [{"role": "user", "content": prompt}],
    }

```

```

}

try:
    r = requests.post(
        "https://api.anthropic.com/v1/messages",
        headers=headers,
        json=body,
        timeout=30,
    )
    data = r.json()
    return data["content"][0]["text"].strip()
except Exception as e:
    logger.error(f"Claude error: {e}")

    return f"Интересный материал по теме: {topic_ru} 📄\n\n#бумага #картон #гофрокартон
#производство #упаковка"

# — Хранилище ожидающих постов (в памяти) —————
pending_posts: dict[str, dict] = {}

# — Отправка поста на одобрение —————

async def send_for_approval(app: Application):
    # Выбираем тему (иногда берём праздничную)
    all_topics = TOPICS + (HOLIDAY_TOPICS if datetime.now().day % 7 < 2 else [])
    topic = random.choice(all_topics)

    logger.info(f"Ищу контент по теме: {topic['ru']}")
    image = search_image(topic["en"])

    if not image:
        await app.bot.send_message(
            ADMIN_CHAT_ID,
            f"⚠ Не нашёл фото по теме «{topic['ru']}». Попробую в следующий раз."
        )
    return

```

```

caption = generate_caption(topic["ru"], image["tags"])

post_id = f"{datetime.now().strftime('%Y%m%d%H%M%S')}_{random.randint(1000,9999)}"

pending_posts[post_id] = {
    "image_url": image["url"],
    "caption": caption,
    "topic": topic["ru"],
}

keyboard = InlineKeyboardMarkup([
    [
        InlineKeyboardButton("✅ Опубликовать", callback_data=f"publish:{post_id}"),
        InlineKeyboardButton("⏭ Пропустить", callback_data=f"skip:{post_id}"),
    ]
])

preview_text = (
    f"📄 *Новый пост на одобрение*\n"
    f"🏷 Тема: {topic['ru']}\n\n"
    f"📝 Подпись:\n{caption}"
)

try:
    await app.bot.send_photo(
        chat_id=ADMIN_CHAT_ID,
        photo=image["url"],
        caption=preview_text,
        parse_mode="Markdown",
        reply_markup=keyboard,
    )
except Exception as e:

```

```

# Если фото не загрузилось — шлём ссылкой
logger.error(f"Не смог загрузить фото: {e}")
await app.bot.send_message(
    chat_id=ADMIN_CHAT_ID,
    text=f"🖼️ [Фото]({image['url']})\n\n{preview_text}",
    parse_mode="Markdown",
    reply_markup=keyboard,
)

```

— Обработка кнопок

```

async def button_handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    action, post_id = query.data.split(":", 1)
    post = pending_posts.pop(post_id, None)

    if not post:
        await query.edit_message_caption("⚠️ Этот пост уже обработан.")
        return

    if action == "publish":
        try:
            await context.bot.send_photo(
                chat_id=CHANNEL_ID,
                photo=post["image_url"],
                caption=post["caption"],
            )
            await query.edit_message_caption(
                caption=f"✅ *Опубликовано в канал!*\n\nТема: {post['topic']}",
                parse_mode="Markdown",
            )

```

```
logger.info(f"Пост опубликован: {post_id}")
```

```
except Exception as e:
```

```
    await query.edit_message_caption(f" ❌ Ошибка публикации: {e}")
```

```
    logger.error(f"Publish error: {e}")
```

```
elif action == "skip":
```

```
    await query.edit_message_caption(
```

```
        caption=f"🔔 *Пропущено*\n\nТема: {post['topic']}",
```

```
        parse_mode="Markdown",
```

```
    )
```

```
    logger.info(f"Пост пропущен: {post_id}")
```

```
# — Команды
```

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    if update.effective_chat.id != ADMIN_CHAT_ID:
```

```
        return
```

```
    await update.message.reply_text(
```

```
        "👋 Бот запущен!\n\n"
```

```
        "Он будет присылать тебе посты *2 раза в неделю* (вт и пт в 10:00).\n\n"
```

```
        "Команды:\n"
```

```
        "/post — получить пост прямо сейчас\n"
```

```
        "/status — статус бота",
```

```
        parse_mode="Markdown",
```

```
    )
```

```
async def manual_post(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    if update.effective_chat.id != ADMIN_CHAT_ID:
```

```
        return
```

```
    await update.message.reply_text("🔍 Ищу контент...")
```

```
    await send_for_approval(context.application)
```

```
async def status(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```

if update.effective_chat.id != ADMIN_CHAT_ID:
    return
await update.message.reply_text(
    f"✅ Бот работает\n"
    f"🕒 Постов на одобрении: {len(pending_posts)}\n"
    f"📅 Расписание: вторник и пятница в 10:00"
)

```

— Заняск

```

def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    # Хэндлеры
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("post", manual_post))
    app.add_handler(CommandHandler("status", status))
    app.add_handler(CallbackQueryHandler(button_handler))

    # Планировщик: вторник (1) и пятница (4) в 10:00
    scheduler = AsyncIOScheduler(timezone="Europe/Moscow")
    scheduler.add_job(
        send_for_approval,
        trigger="cron",
        day_of_week="mon,wed",
        hour=07,
        minute=0,
        args=[app],
    )
    scheduler.start()
    logger.info("Планировщик запущен: пн и ср в 07:00 МСК")

    app.run_polling(drop_pending_updates=True)

```



```
if __name__ == "__main__":  
    main()
```